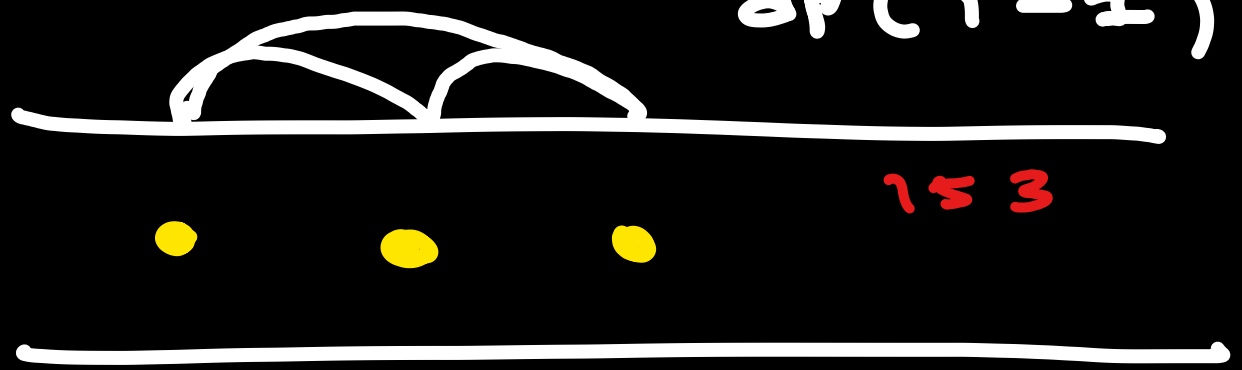


1 5 7 7 → 167

167 → 16

16 → 1

sumdigits(N/10)

153 → 15

15 → 1 } /10

sum digits (153)

153 mod 10 + sumdigits(153/10)

153 mod 10 + sumdigits(15)

153 mod 10 + 15 mod 10 + sumdigits(15/10)

153 mod 10 + 15 mod 10 + sum digits (1)

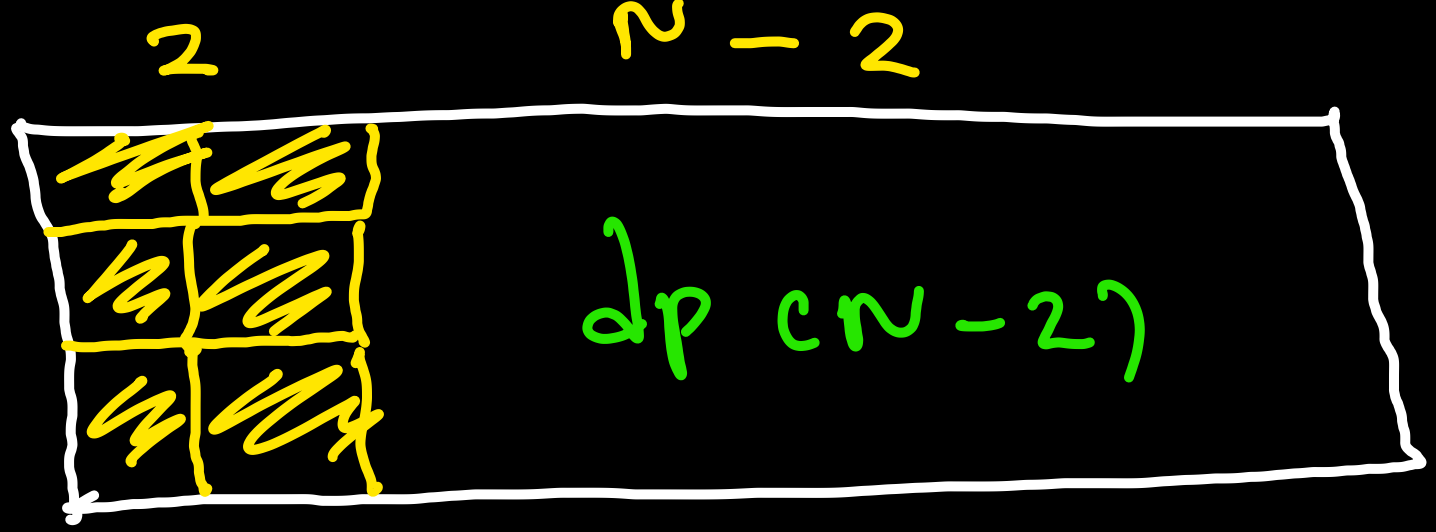
153 mod 10 + 15 mod 10 + 1 mod 10

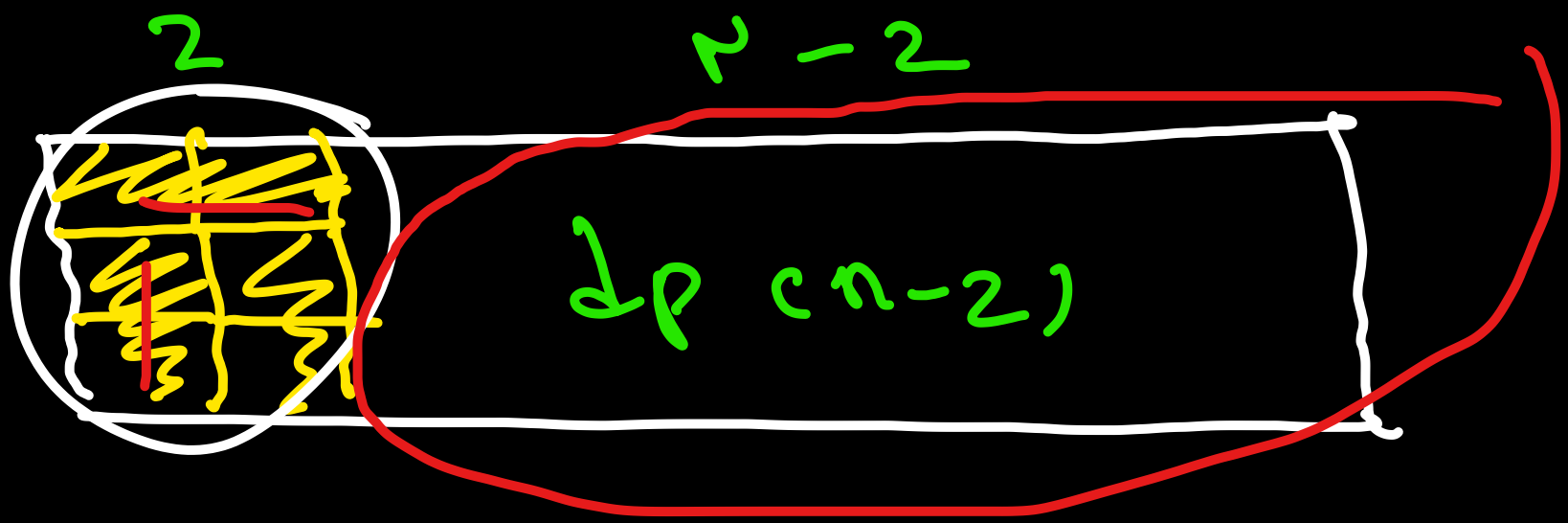
3 + 5 + 1 = 9



dp(n) = bc

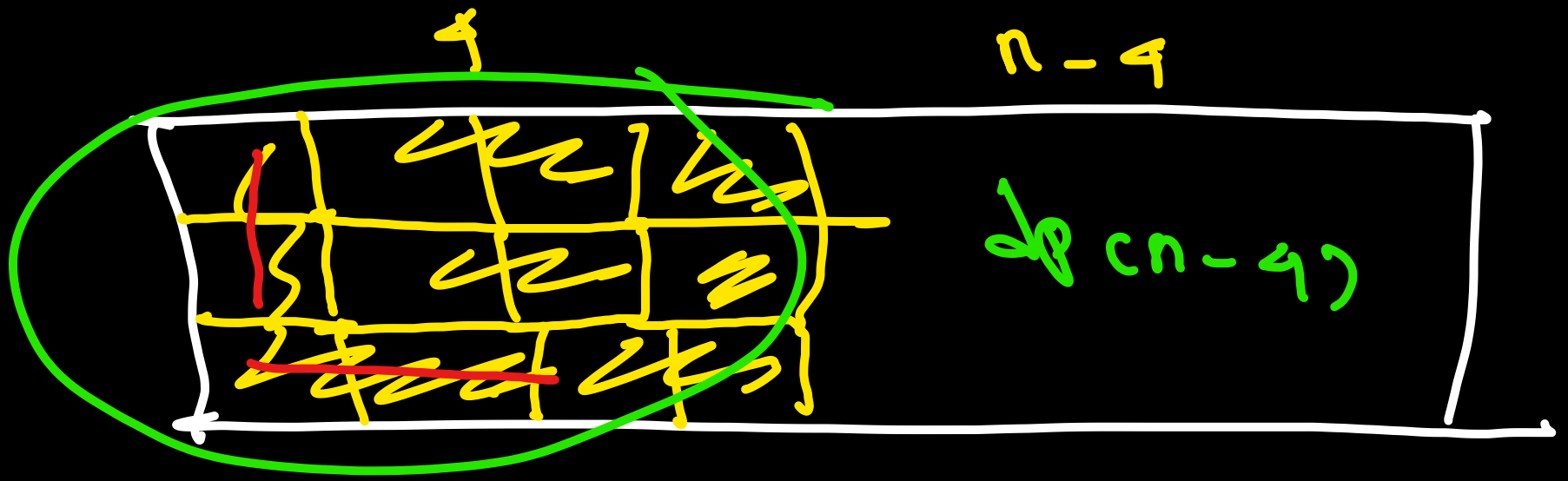
Mengisi: 3 × n





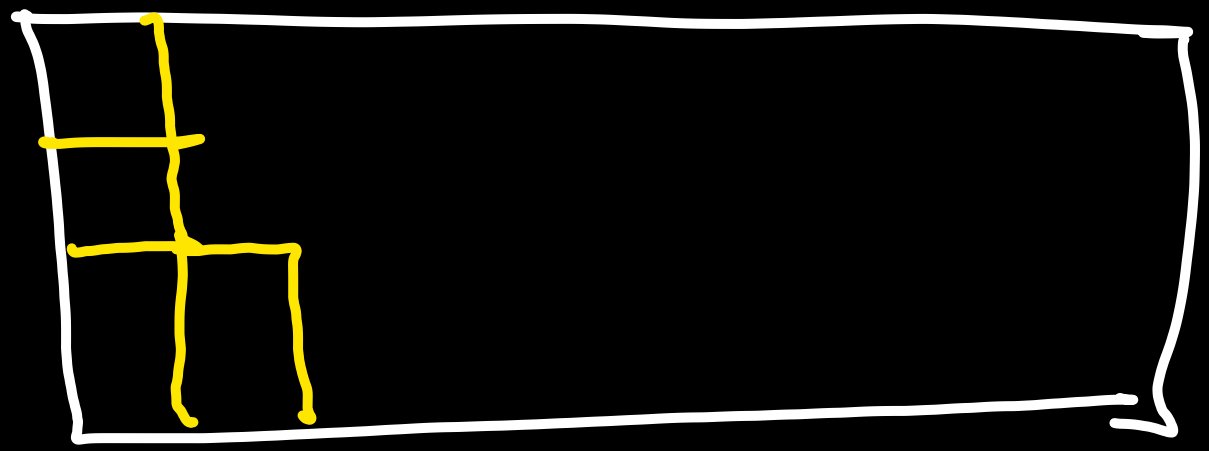
dp (n-2)

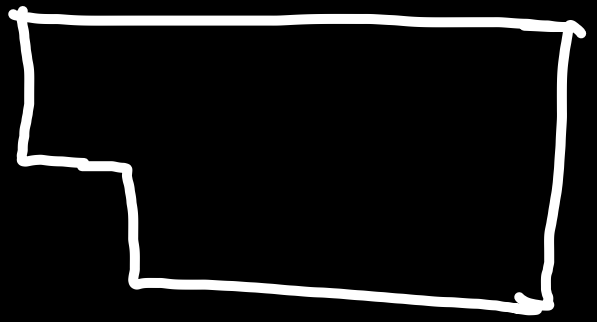
3 dp (n-2)



$$dp(n-4)$$

$$dp(n) = dp(n-2) * 3 + 2\,dp(n-4)$$





g(n)

f(n) ⇒ f(n-2) ⇒ cout ~

f(5) ⇒ f(1) ⇒ 1 ⇒ 5

f(3) ⇒ 3

f(2) ⇒ 2

f(1) ⇒ 1

(1) DnC → Worst Time Complexity

(Recursive)

Best Space Complexity ⇒ Memory

(2) Dp →

(Recursive)

Best Time Complexity

Worst Space Complexity

Memo[100001]

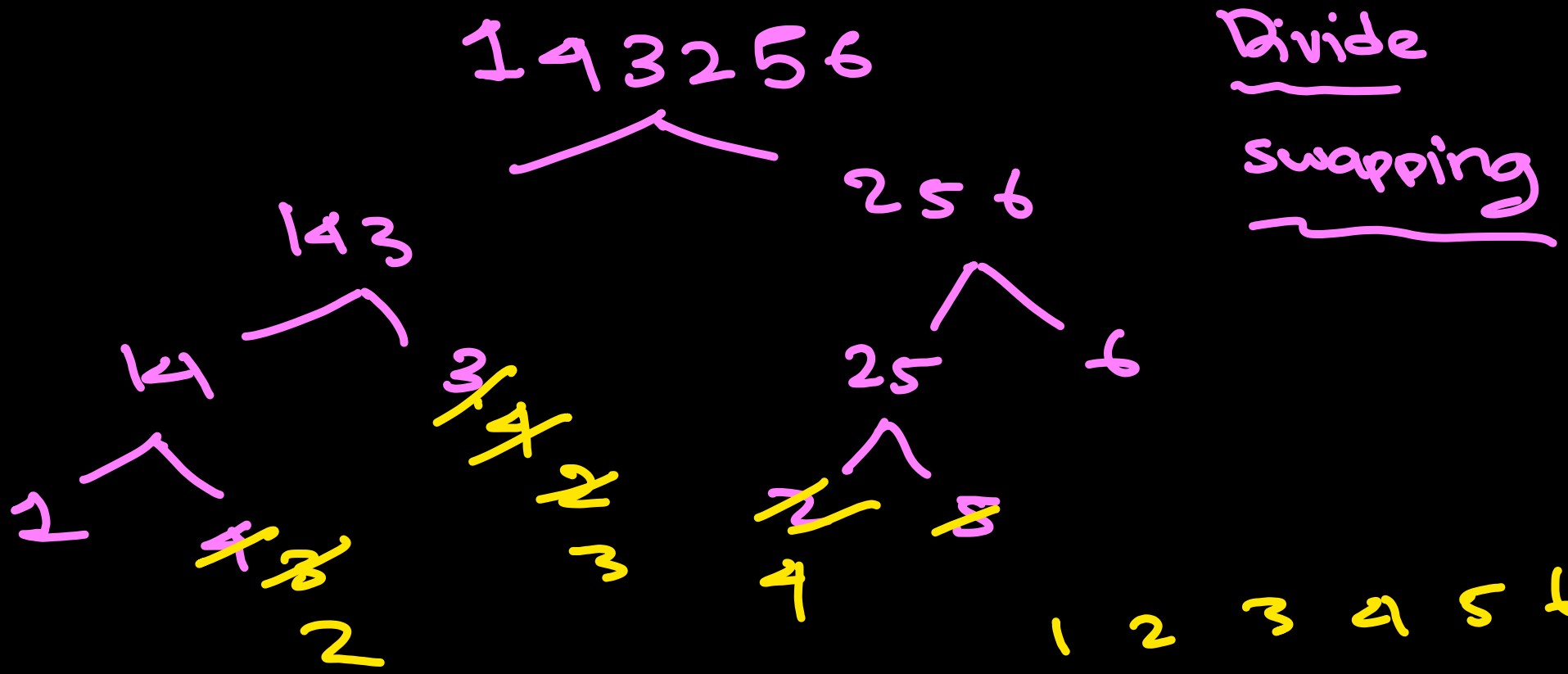
DnC Vs Dp

DnC ↙ No Memoization

Dp ↘ Memoization

* Merge Sort ⇒ DnC → 1. Divide 2. conquer 3. Combine/ merge

→ gak butuh memo ?



Divide

swapping

Prosedur Void

Merge Sort ⇒ ~~Return Value~~ Tdk harus ?

Merge ()

Fibonacci ⇒ Return Value / Mendapat nilai ⇒ Memo (DP)

Memo [10001]

Recursive :

(1) Base case

(2) Rekurens

$$f(n) = \underbrace{f(n-1) + f(n-2)}_{\text{Rekurens}}$$

$$\underbrace{f(1) = 1,\ f(2) = 1}_{\text{Base case}}$$

$$(a + b) \bmod n$$

$$(a \bmod n + b \bmod n) \bmod n$$

$$\underbrace{\left(\underbrace{10^9 + 8}_{a} + \underbrace{10^9 + 9}_{b}\right)}_{} \bmod$$

$(10^9 + 8 \mod 10^9+7) + (10^9+9 \mod 10^9+7)$

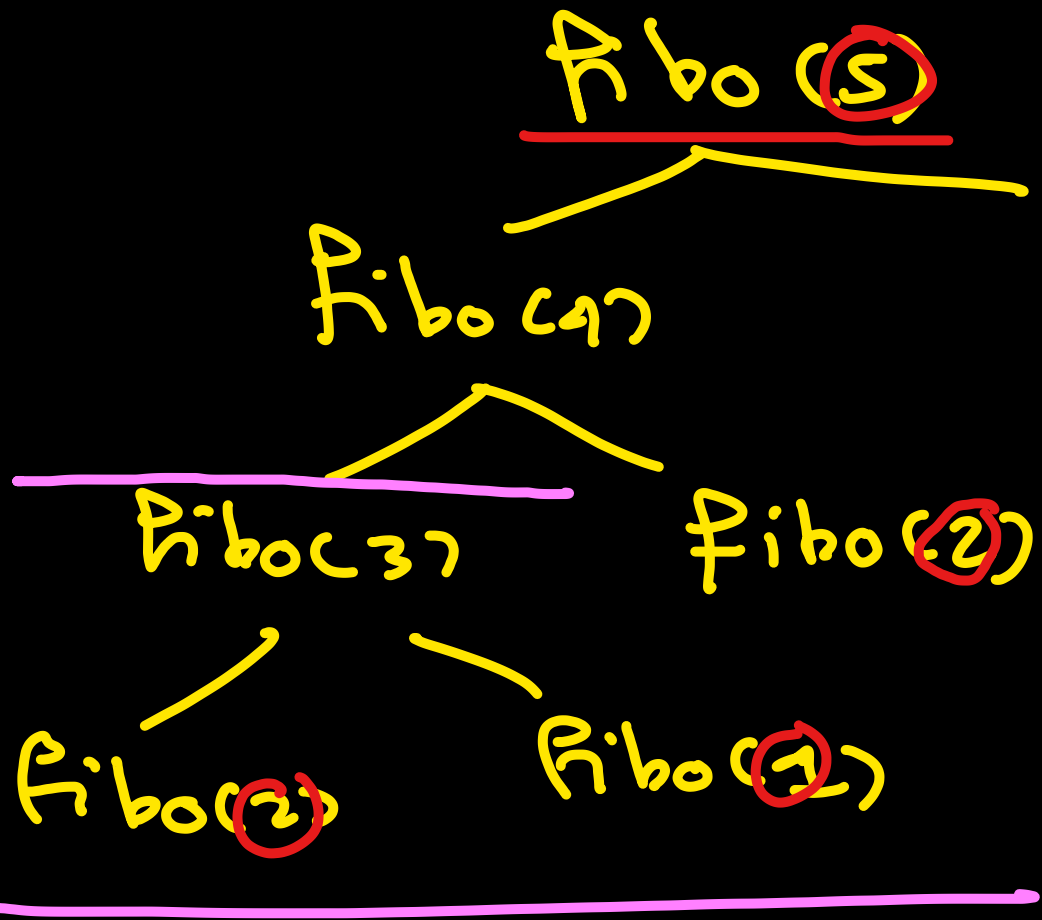
$= 1 + 2 = 3 \mod 10^9+7$

$= 3$

Dynamic Programming

- Top down
- Bottom up

Top down → Function + Memoization (umumnya)



Bottom up ⇒ Tabulation / array

fibo(5) = . . .

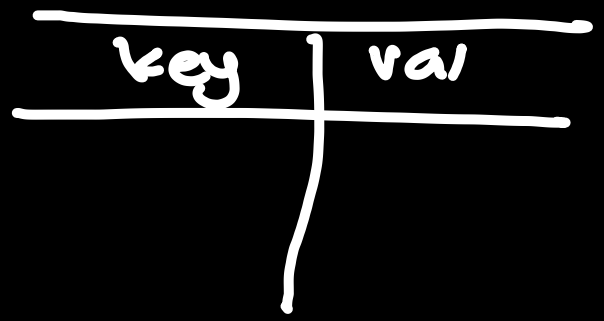
Base case → Problem

```
fibo[1] = 1
fibo[2] = 1
for n from 3 to 5 ->
   fibo[n] = fibo[n - 1] + fibo[n - 2]
```

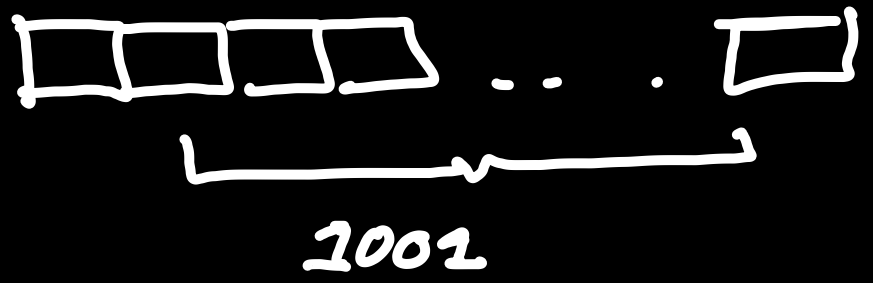
Base case

Bottom - up

* array ⟺ hash table / Map



memo[1001] =



* Buat Model → dpcm = ... - ..
* Base case = dpc1) = ... , dpc2) = ..

(1) Optimasi: Min-Max + Greedy
(2) Combinatorics
(3) Traverse ⇒ Movement

Abstraksi ⇒ decomposing.



→

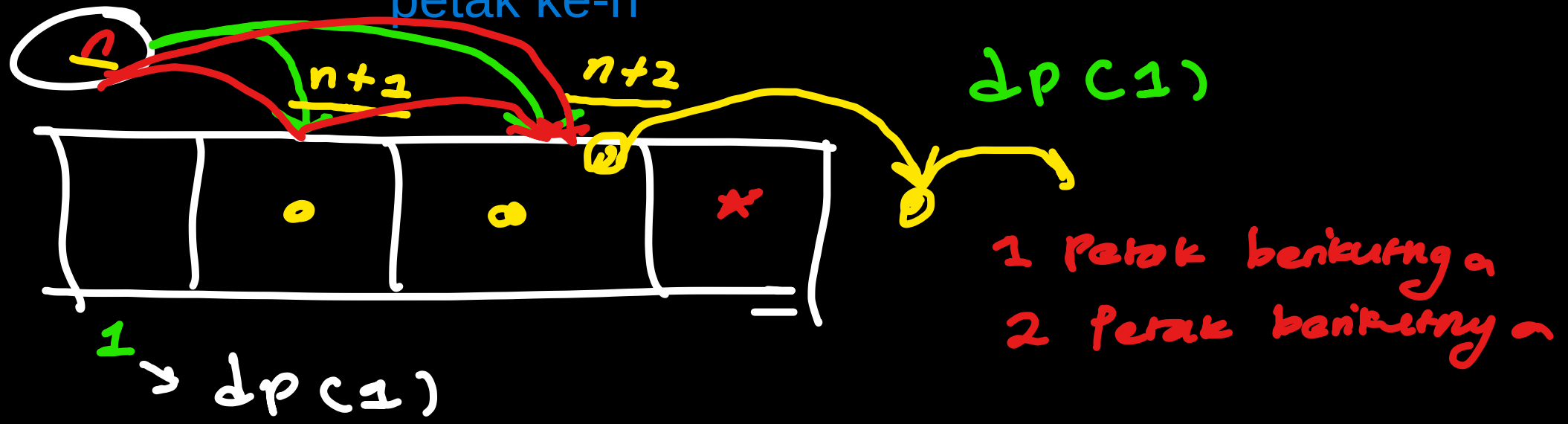
```
Rumah {
  atap;
  jendela;
  pintu;
  dinding;
}
```



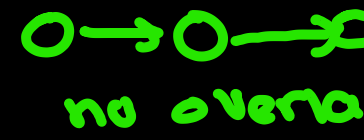
Pulpen

Bottom-up

dp(n) = Banyak koin max, jika kita mulai bergerak dari petak ke-n



Max (Jalan mana yang menghasilkan min max
Loncat 1 petak,
Loncat 2 petak)



$dp(n) = \max(dp(n+1), dp(n+2)) + \text{item}[i]$

@ → 1 , • → 0 , + → -1

Pilih jarak terjauhnya.

{ Max (@, •) → max(1, 0) = 1
Max (•, +) → max(0, -1) = 0